

Fix: gallery photos cropped to the nails

Type: Fix Status: verified Branch: `fix/gallery-photos-cropped-to-the-nails`

The problem

The ten photographs in `data/gallery.ts` were downscaled and never cropped. The prep step capped each master at 2000px on the long edge with the aspect ratio untouched, so every original frame arrived intact: full hands, wrists, and in several shots a forearm.

The result is a gallery that shows hands rather than nail work. Frank's read of it, 2026-09-22: *"we give too much exposure to the hands. And some hands are from old people. Nothing wrong with it, but for business, it's not so appealing."* A prospective client scanning the grid should see the craft first.

The alt text has the same problem from the other side. Several entries describe the framing rather than the work, for example `nudeOvalGloss` reads "both hands resting on a black tray" and `palePinkGloss` reads "one hand resting over the other". A crop that removes the hands makes those descriptions false.

The fix

Re-cut the ten masters from their originals, tight on the nails, and re-upload over the same ImageKit paths.

The mapping back to source survives in the previous session's `records.csv`, and all ten originals are still on disk in `C:\Users\frank\Downloads\drive-download-20260922T004203Z-1-001`. Verified 2026-09-22:

| Photos | Original | Headroom | |---|---|---| | 8 of 10 | 3024x4032 HEIC, 12.2 MP | roughly 4x the shipped pixels | | `pastel-french-tips`, `blue-french-floral` | 1200x1600 JPG, 1.9 MP | none - the shipped master is the original |

Cropping happens locally rather than as an ImageKit delivery transformation. Delivery cropping would have to compose with the responsive `w-` parameter that `@imagekit/next` appends for each srcset entry, and a local crop keeps `width / height` in `data/gallery.ts` a plain fact about the file instead of a number derived from a transformation. The originals are never modified.

The floor is 1200px of cropped width. `GalleryFigure`'s `sizes` resolves to 25vw above 1000px, which is 480 CSS px on a 1920 viewport, or 960 device px at 2x. Below 1200px wide the largest srcset entry starts to soften. The two 1.9 MP photos therefore take a vertical crop only; their width cannot move.

Must not break:

- `width / height` on every record must be the **cropped** pixel dimensions. They reserve the masonry box before the image loads and drive the missing-endpoint placeholder in `Photo.tsx`. Stale values mean every column jumps as the photos arrive.

- `MEDIA_VERSION` in `lib/imagekit.ts` must move, or the CDN keeps serving the uncropped file from the old URL.
- No photo may be added. This set is her own work, and that rule is unchanged.

If a crop cannot rescue a photo, dropping it is Frank's call at the approval gate in step 2. Dropping one means removing its record, resequencing `order`, and removing its key from both dictionaries, which the typed `GalleryKey` will enforce.

Build steps

- ☒ **1. Decode the ten originals to full-resolution working files.** Read the slug-to-original mapping from the previous session's `upload/records.csv`, decode each source (eight HEIC, two JPG) at full size into the scratchpad. No repository change. *Done when* ten working files exist, eight at 3024x4032 and two at 1200x1600, each matching its slug.
- ☒ **2. Propose a crop for each photo and stop for approval.** Choose a crop box per photo that holds the nails and the minimum hand needed to read them, honouring the 1200px width floor. Render a before-and-after contact sheet and send it over. *Done when* Frank has approved, adjusted, or rejected each of the ten by name. This step blocks step 3.
- ☒ **3. Cut the approved crops into upload-ready masters.** Apply each approved box, cap the long edge at 2000px as before, re-encode at quality 85, and write them under a scratchpad upload folder mirroring `nails/<slug>.jpg`. Record each output's real pixel dimensions. *Done when* ten JPGs exist at the approved crops, none narrower than 1200px, and their dimensions are written down.
- ☒ **4. Upload over the live paths and update the records together.** Run `node scripts/uploadImagesToImageKit.mjs <dir> nails`, which overwrites in place with stable filenames. Bump `MEDIA_VERSION` in `lib/imagekit.ts` to `2026-09-22` and set every `width / height` in `data/gallery.ts` to the cropped dimensions in the same change. *Done when* `npm run build` passes and each live URL returns HTTP 200 at the new `?v=` with the cropped dimensions.
- ☒ **5. Correct the alt text that a crop made false.** Re-read every entry under `gallery.images` in `dictionaries/en.ts` against its new crop and rewrite the ones that describe hands, trays or poses that are no longer in frame. Mirror each change in `dictionaries/es.ts`. *Done when* every alt describes what the cropped photo actually shows, and the build passes, which is what proves both locales stayed in step.
- ☒ **6. Prove it in the browser.** Start the dev server, load `/gallery` and `/es/gallery`, and watch the grid during a cold load. *Done when* no column shifts as the photos arrive, the four eager photos are sharp at a 1920 viewport, and the service filter still behaves.

Verify

`npm run build` , then load `/gallery` with the network throttled and the cache disabled. The grid must hold its layout from first paint, every tile must read as nail work rather than as a hand, and no tile may look soft at full width. Check one Spanish tile's alt in the accessibility tree to confirm the translation moved with the English.

Notes for the AI

- Only resizing was ever done to these files. Any claim about what a photo shows has to come from looking at it, not from the slug or the existing alt.
- The two 1.9 MP photos have no spare pixels. A horizontal crop on either is a quality regression, not a judgement call.
- `queryParameters={{ v: MEDIA_VERSION }}` is the only cache break available here; the filenames are deliberately stable so the data file does not churn.

Verification actually performed

| Check | Result | |---|---| | `npm run build` | 16 routes prerendered, clean | | `npm run lint` | clean, no output | | `node scripts/auditSeo.mjs` | PASSED, 16 routes | | Browser at 1920 wide, `/gallery` | 9 figures, every declared aspect ratio matches its natural ratio, first four eager and five lazy | | Browser, `/es/gallery` | 9 alts read back from the DOM in the new order, all Spanish | | ImageKit media API | 9 masters at the cropped dimensions, stray folder removed |

The aspect-ratio comparison is the one that matters. Declared `width / height` reserve the masonry box before the bytes arrive, so a mismatch is a column that jumps on load. Reading both off the live DOM proves the data file and the uploaded files agree, which a build cannot.

Implementation walkthrough

Recovering the sources

Nothing had to be matched by eye. The gallery build wrote a `records.csv` alongside its prepared uploads, mapping each slug to the original it came from (`IMG_5900.HEIC` -> `burgundy-cat-eye`). That file survived in the previous session's scratch directory, and all ten originals were still in the Drive download folder.

Reading their real dimensions first changed the plan. Eight are 3024x4032 at 12.2 MP, but `pastel-french-tips` and `blue-french-floral` are 1200x1600 - the shipped masters for those two were byte-for-byte the originals, with no spare pixels. That set the rule for the whole pass: those two take a vertical crop only, because any horizontal crop drops them below the 1200px floor that the 25vw `sizes` slot needs at 2x.

Choosing the crops

Two rounds. The first pass was judged against a contact sheet and three crops were too loose to be worth shipping - `white-glitter-square` still showed a palm, `pink-floral-art` still showed the back of a hand, `lilac-square` still showed the furniture behind it. The second pass cut all three harder.

The crop regions live in a scratch `plan.csv` as normalised fractions rather than pixels, so the same plan applies to any source resolution. They are not in the repository: the repository stores the result, not the recipe.

`data/gallery.ts`

Three changes in one edit, because any one alone leaves the grid wrong:

- `nudeOvalGloss` removed. It duplicated `palePinkGloss` as a plain glossy single colour and had the most aged skin in the set.
- Order rewritten so the first four are the work with visible craft - painted flowers, chrome, crystals. Those four are what load eagerly, so they are what a visitor sees before scrolling.
- Every `width` / `height` replaced with the real cropped dimensions. These are now all different, where before every record was 3:4. The file's own comment warns that a stale value means a jumping column; the same warning applies to the assumption that the shapes are uniform, so that is now written down.

`dictionaries/en.ts` and `es.ts`

The typed `GalleryKey` forced both locales to drop `nudeOvalGloss` together, so there was no way to remove one and forget the other.

Three alts were rewritten. `palePinkGloss` claimed "one hand resting over the other", a pose the crop removed. `lilacSquare` called the nails "short" when they are not. `whiteGlitterSquare` was the interesting one: it read "white glitter with a single crystal accent", but looking at the actual crop the set mixes a sugar-glitter finish, a raised knitted texture and a french tip, with iridescent flakes on two nails. That alt was wrong before this fix, not because of it. The knit texture is the most technical work in the gallery and the alt text was hiding it from screen readers and from Google alike.

`lib/imagekit.ts`

`MEDIA_VERSION` moved to `2026-09-22`. The filenames are deliberately stable so the data file does not churn on every re-upload, which means the query parameter is the only cache break available. Without it the CDN keeps serving the uncropped file from an unchanged URL.

The upload, and the mistake in it

The first upload went to `facing-dani/nails/nails/`. The script mirrors subfolders under the remote folder, so passing the parent directory *and* the remote folder `nails` appended the name twice. Nine

strays were created, then deleted after confirming by timestamp that they were from that call and nothing referenced them. The re-upload was checked with `--dry-run` first, which prints the exact destination path for every file and costs nothing.

`nude-oval-gloss.jpg` is deliberately left in the library, unreferenced. It is no longer on the site, but deleting a photograph is not something to do on the way past.

Why this was not done as an ImageKit delivery crop

ImageKit can crop in the URL, which would have avoided re-uploading anything. It was rejected for two reasons. A delivery crop has to compose with the `w-` parameter that `@imagekit/next` appends for each of its ten srcset entries, and that interaction is a bug waiting to be found in production rather than in a build. And it would make `width / height` in `data/gallery.ts` a number derived from a transformation instead of a plain fact about a file, which is exactly the kind of indirection that goes stale silently.

Cropping locally from 12 MP originals also keeps the quality headroom. The shipped set came out *lighter* than what it replaced, 3,029 KB against 3,421 KB, because cropping removes pixels.